

Transformer：注意力架構與預訓練

位置、遮罩、多頭注意力與 BERT

書本第 15 章

學習重點與成果

- Transformer 結構、位置資訊與 Q/K/V。
- 多頭注意力、遮罩、殘差與翻譯。
- BERT 預訓練、微調與模型比較。

成果：能手算小型注意力，追蹤維度並辨識未來資訊洩漏。

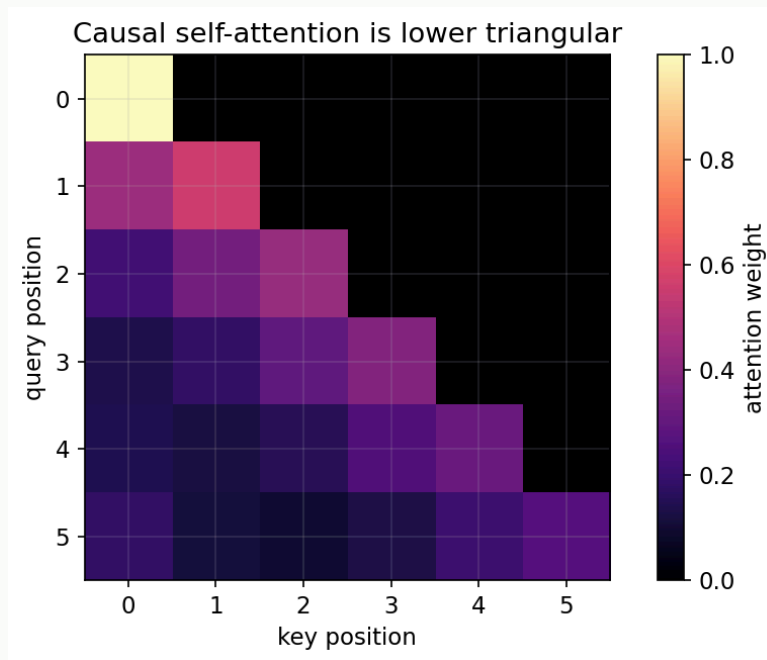
 **配套 Notebook**： `programs/notebooks/18_Transformer_注意力架構與預訓練.ipynb` 。
本檔提供公式驗算、機制實驗與結果圖；框架片段及完整模型案例另依頁面說明閱讀。

程式實驗與實際輸出

```
scores[future_mask] = -inf  
weights = softmax(scores)
```

觀察：遮罩後未來位置的注意力總量為 0，權重矩陣呈下三角。

參考程式：`programs/notebooks/18_Transformer_注意力架構與預訓練.ipynb`

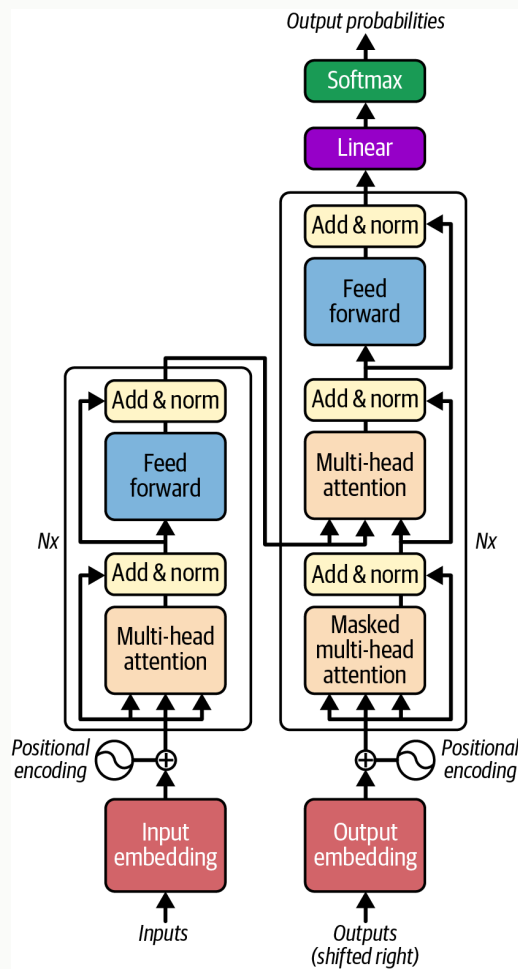


三種 Transformer 家族

架構	可見上下文	常見任務
Encoder-only	左右兩側	分類、抽取
Decoder-only	目前與先前位置	逐 token 生成
Encoder-decoder	來源全文與目標前綴	翻譯、摘要

家族名稱描述資訊流，不直接代表模型大小或品質。

原始 Transformer 的完整架構



- 左側 encoder 讀取來源序列。
- 右側 decoder 讀取位移後的目標前綴。
- 中間 cross-attention 接收 encoder 的表示。
- 兩側都有殘差、正規化與 FFN。

序列沒有循環，仍需要位置資訊

Self-attention 本身依內容配對，不能單靠 token 集合知道順序。

常見方式：可學習位置向量，或固定 $\sin/\cos$ 編碼。

把位置向量與 token embedding 相加時，兩者最後一維必須相同。

可學習位置向量

```
class PositionalEmbedding(nn.Module):
    def __init__(self, max_length, d_model):
        super().__init__()
        self.pos = nn.Parameter(torch.randn(max_length, d_model) * 0.02)
    def forward(self, X):
        return X + self.pos[:X.size(1)]
```

使用 `torch` 與 `torch.nn` ；X 為 `[B,T,D]` 。輸入長度不可超過表長。

固定位置編碼的概念

$$PE_{p,2i} = \sin\left(p/10000^{2i/d}\right)$$
$$PE_{p,2i+1} = \cos\left(p/10000^{2i/d}\right)$$

每個維度使用不同頻率；位置 0 的偶數維為 0、奇數維為 1。

可以計算更長位置，不代表模型一定能可靠泛化到更長序列。

Q、K、V 的分工

- Query：目前位置要尋找的特徵。
- Key：各候選位置用來比對的特徵。
- Value：實際加權整合的內容。

Self-attention 的三者來自同一序列的不同線性投影；cross-attention 的來源不同。

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

矩陣	單一 head 的形狀
Q、K、V	$L_q \times d_k$ 、 $L_k \times d_k$ 、 $L_k \times d_v$
Scores、輸出	$L_q \times L_k$ 、 $L_q \times d_v$

Softmax 沿 key 位置計算。

點積注意力手算

令 $q = [1, 0]$, $k_1 = [1, 0]$, $k_2 = [0, 1]$, $d_k = 2$ 。

分數為 $[1/\sqrt{2}, 0]$, Softmax 約為 $[0.670, 0.330]$ 。

若 $v_1 = [2, 0]$, $v_2 = [0, 4]$, 輸出約為 $[1.340, 1.320]$ 。

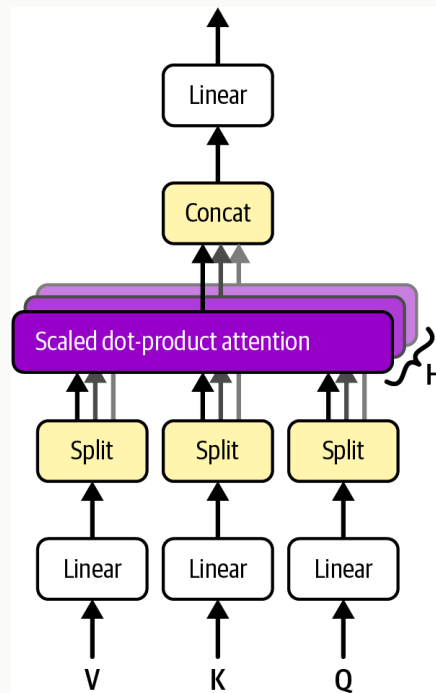
活動：交換兩個 value 後，分數與輸出各會如何改變？

四行看懂 attention 核心

```
import torch.nn.functional as F
scores = Q @ K.transpose(-2, -1) / Q.size(-1)**0.5
scores = scores.masked_fill(blocked_mask, -torch.inf)
weights = F.softmax(scores, dim=-1)
context = weights @ V
```

此自寫程式的 `blocked_mask=True` 表示禁止關注；每列至少保留一個有效 key。

多頭注意力分別學習不同投影



每個 head 產生一組 context，串接後再經輸出投影。

多頭的維度追蹤

設 $B = 2, L = 10, d_{model} = 64, h = 4$ 。

階段	形狀
投影後	[2, 10, 64]
拆分 head	[2, 4, 10, 16]
每頭分數	[2, 4, 10, 10]
合併後	[2, 10, 64]

64 必須可被 4 整除。

兩種遮罩處理不同問題

遮罩	禁止看什麼
Padding mask	為湊 batch 長度而增加的補值
Causal mask	當前 token 之後的位置

翻譯 decoder 還要在 cross-attention 中排除來源端 padding。

不同 API 的布林 mask 意義可能相反，不能直接套用同一變數。

Causal mask 手算

四個位置，1 表示允許關注：

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

訓練時可平行計算所有位置，但每個位置只能用自己的前綴。

Feed-forward network 與殘差

FFN 對每個位置套用同一組非線性轉換。

$$FFN(x) = W_2 \phi(W_1 x + b_1) + b_2$$

Attention 混合不同位置的資訊；FFN 轉換每個位置內的特徵。

殘差連接要求輸入與輸出的 d_{model} 一致。

LayerNorm 手算：四維例子

對單一 token 向量 $[2, 4, 4, 6]$ ：

平均 $\mu = 4$ ，變異數 $\sigma^2 = (4 + 0 + 0 + 4)/4 = 2$ 。

忽略 ϵ 且 $\gamma = 1, \beta = 0$ ，結果約為
 $[-1.414, 0, 0, 1.414]$ 。

不是跨整個 batch 計算這組統計量。

Pre-norm 與 post-norm

原始架構把 norm 放在殘差相加之後；其他設計可放在子層之前。

設計	簡化表示
Post-norm	$LN(x + F(x))$
Pre-norm	$x + F(LN(x))$

讀碼時確認 norm 的位置，不因都叫 Transformer 就假設相同。

翻譯 Transformer 的遮罩位置

```
# src、tgt 都已加上 token 與位置 embeddings
model = nn.Transformer(d_model=64, nhead=4,
                        num_encoder_layers=2, num_decoder_layers=2,
                        batch_first=True)
output = model(src, tgt, tgt_mask=causal_blocked,
               src_key_padding_mask=src_pad,
               tgt_key_padding_mask=tgt_pad,
               memory_key_padding_mask=src_pad)
```

src_pad、tgt_pad 中 True 表示 padding；tgt 需使用位移後的目標前綴。

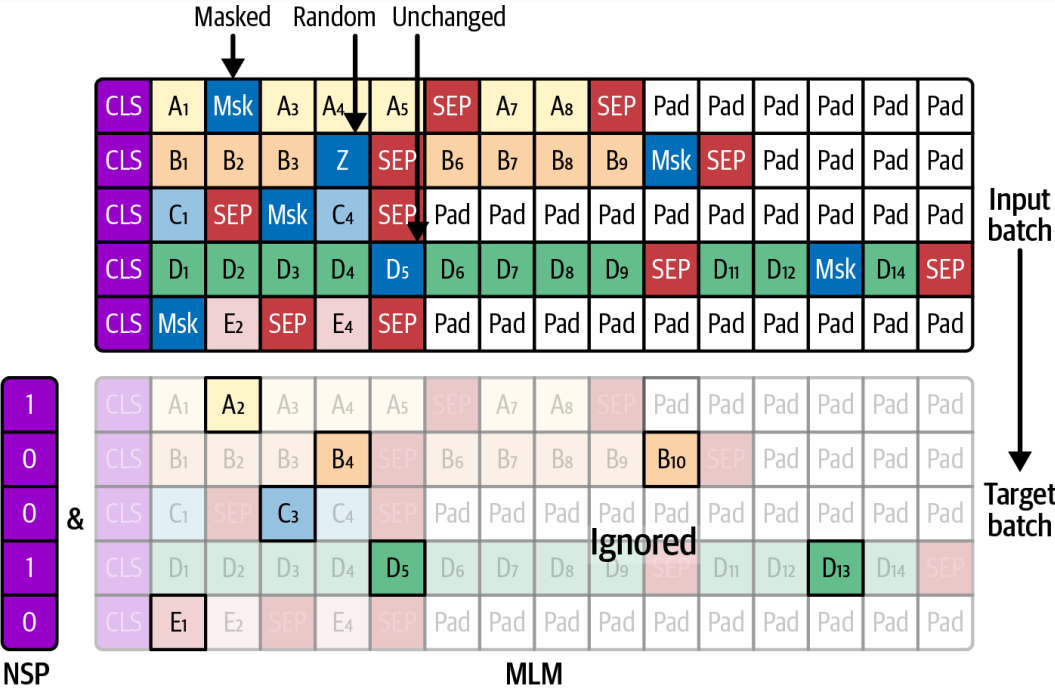
Encoder-only 模型與 BERT

BERT 用雙向上下文表示，常用於分類與資訊抽取。

預訓練先學通用語言表示，微調再對應任務標籤。

只用 encoder 並不等於沒有任務頭；不同任務的輸出形狀不同。

BERT 的預訓練目標



MLM 預測選中的 token ；原始 BERT 也使用 NSP 判斷句子配對。

MLM 的遮蔽規則

原始 BERT 選約 15% token 作預測目標。

在被選中的位置中：約 80% 換成 [MASK]、10% 換隨機 token、10% 保持原字。

損失只計算被選中的位置；未換掉的目標仍需預測。

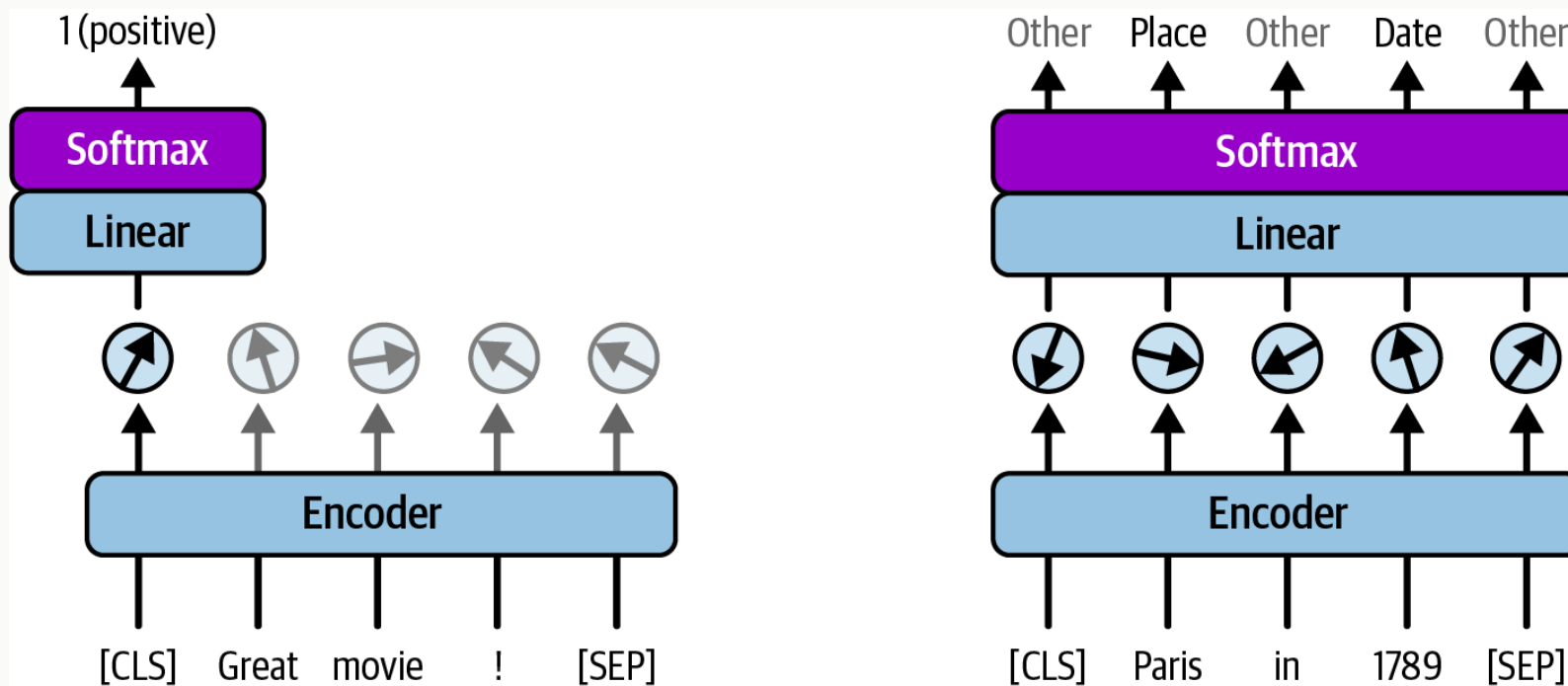
這是原始設計，後續模型可採不同訓練配方。

小型 BERT：區分隨機初始化與預訓練

```
from transformers import BertConfig, BertForMaskedLM
config = BertConfig(vocab_size=30522, hidden_size=128,
                    num_hidden_layers=2, num_attention_heads=4,
                    intermediate_size=512, max_position_embeddings=128)
model = BertForMaskedLM(config)
```

這會建立隨機權重，尚未懂語言；`from_pretrained(...)` 才會載入既有權重。

分類頭與逐 token 任務頭



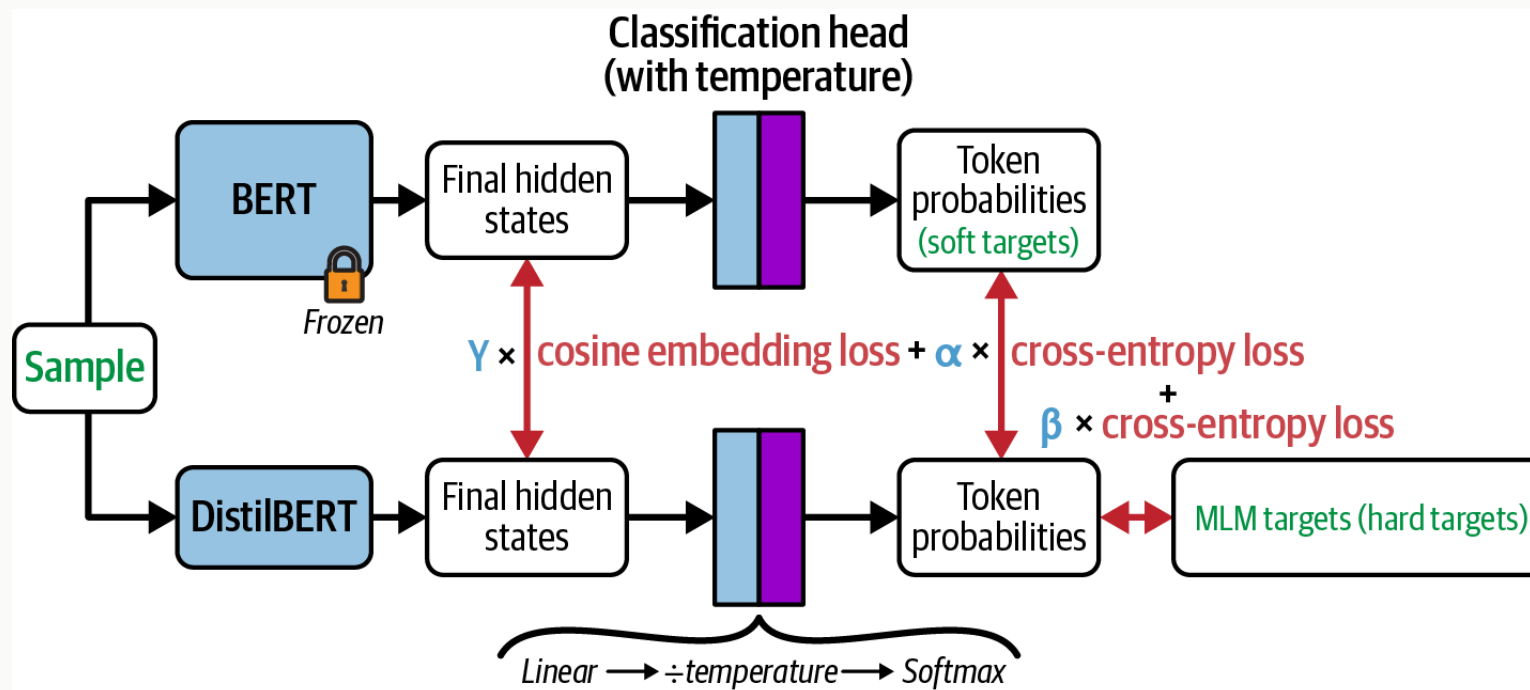
句子分類輸出每句一組標籤；NER 或 POS 則要對齊每個有效 token。

問答任務的輸出設計

任務	輸出
句子分類	一個類別
Token classification	每個token一個類別
抽取式問答	答案開始與結束位置
多選問答	每個候選答案的分數

抽取式問答通常從提供的文本找片段，與自由生成回答不同。

DistilBERT 的知識蒸餾



學生同時學任務訊號與教師行為；壓縮模型需重新衡量品質與成本。

Encoder 模型的改良方向

模型	教材中的主要改良
RoBERTa	調整資料量與訓練配方，移除NSP
ALBERT	分解embedding與跨層參數共用
ELECTRA	辨識被替換的token
DeBERTa	分開處理內容與位置資訊

課堂活動：找出注意力錯誤

檢查下列三個設計：

1. Softmax 沿 batch 軸。
2. Decoder 訓練時可以看到目標下一個 token。
3. Cross-attention 使用 decoder 的輸出當 Q，encoder 輸出當 K/V。

交付：前兩者的修正與第三者的尺寸表。

離堂檢核

- 自注意力與交叉注意力的 Q/K/V 分別來自哪裡？
- 為何有 causal mask 仍可平行訓練？
- BERT 的 15% 與 80% 的母數分別是什麼？
- 建立 config 是否等於載入預訓練知識？

課後程式與延伸閱讀

- 作者第 15 章 Notebook (`15_transformers_for_nlp_and_chatbots.ipynb`)：先執行 Setup，再定位本課指定區段。

執行前確認資料、套件與運算資源。

Notebook 導讀：可執行的機制與驗收

實驗	驗收證據
Encoder／decoder 與多頭	Q/K/V 形狀、FFN、殘差、norm
Causal 與 padding mask	改未來不改過去；padding 權重為零
MLM、分類、token 與 QA 頭	受監督位置、輸出形狀、合法 span
蒸餾	固定教師分布、學生 KL 下降

權重隨機或只優化玩具 logits；不稱為已預訓練 BERT／DistilBERT。
完整模型 API 仍作讀碼；小型實驗提供可核對的公式與資料流。